



AI with Rav

Learn AI the way you actually think.



DAY 13 of 30

MACHINE LEARNING

K-Means — finding groups with no labels

WHAT YOU'LL LEARN TODAY

- › What "no labels" means (unsupervised learning)
- › The teacher-in-the-playground trick
- › Pick centers, join nearest, move to middle
- › Why it repeats until nobody switches
- › How to choose K (the elbow)



K-Means — finding groups with no labels

In One Line: A new teacher walks into a noisy playground. No one told her who's friends with whom. But she finds the natural groups all by herself — by picking a few spots and telling each kid to stand near the closest one. That's K-Means.

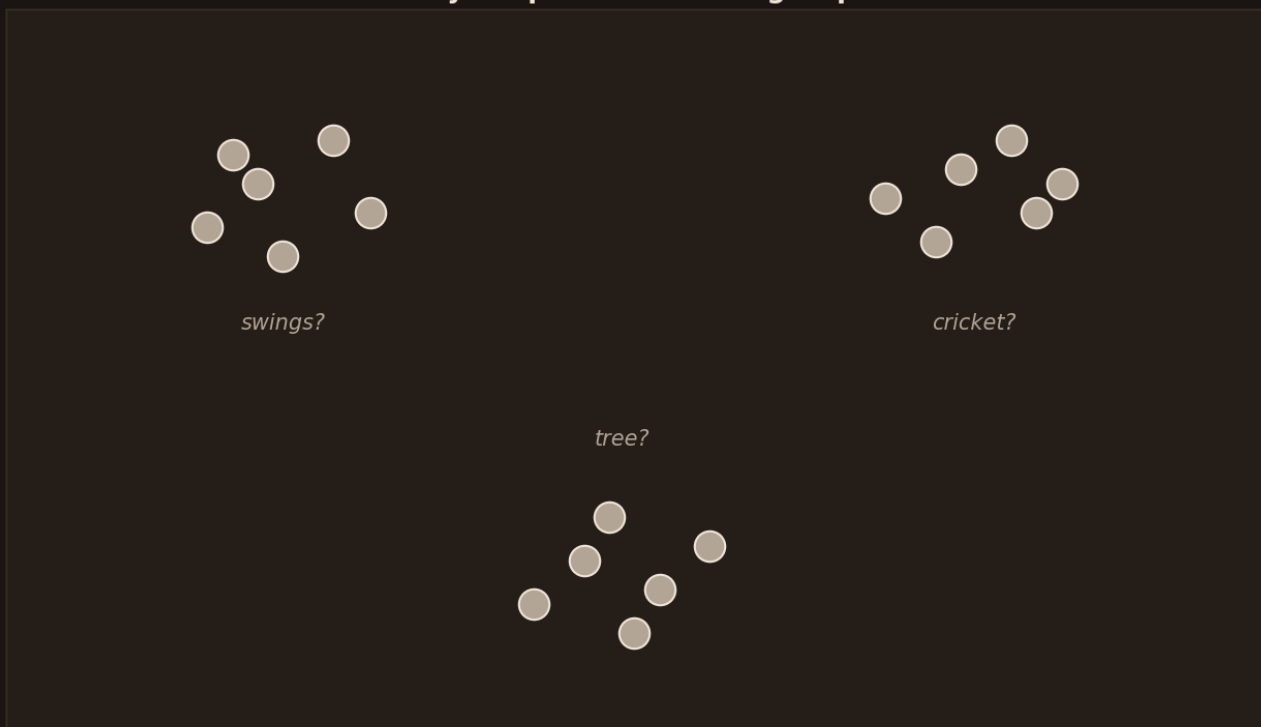
Start here: something totally new — no answers given

Every algorithm so far had a **teacher with an answer key**: this email is spam, that one isn't. The machine learned by copying the right answers. That's called **supervised learning** (someone supervises with labels).

But today is different. Imagine you just have a **pile of data and NO labels at all** — nobody tells you the groups. Can a machine still find them? Yes — and this is called **unsupervised learning**. **K-Means** is the most famous example.

The playground with no name tags

A playground of kids — nobody is labelled.
Can you spot the natural groups?



A playground full of kids, none of them labelled. Yet your eye already sees natural groups — near the swings, near the cricket pitch, under the tree. K-Means teaches a machine to see them too.

A new teacher walks in at recess. Kids are scattered everywhere — no name tags, no list of friend-groups. But look closely: they naturally clump together.

- A cluster near the **swings**.

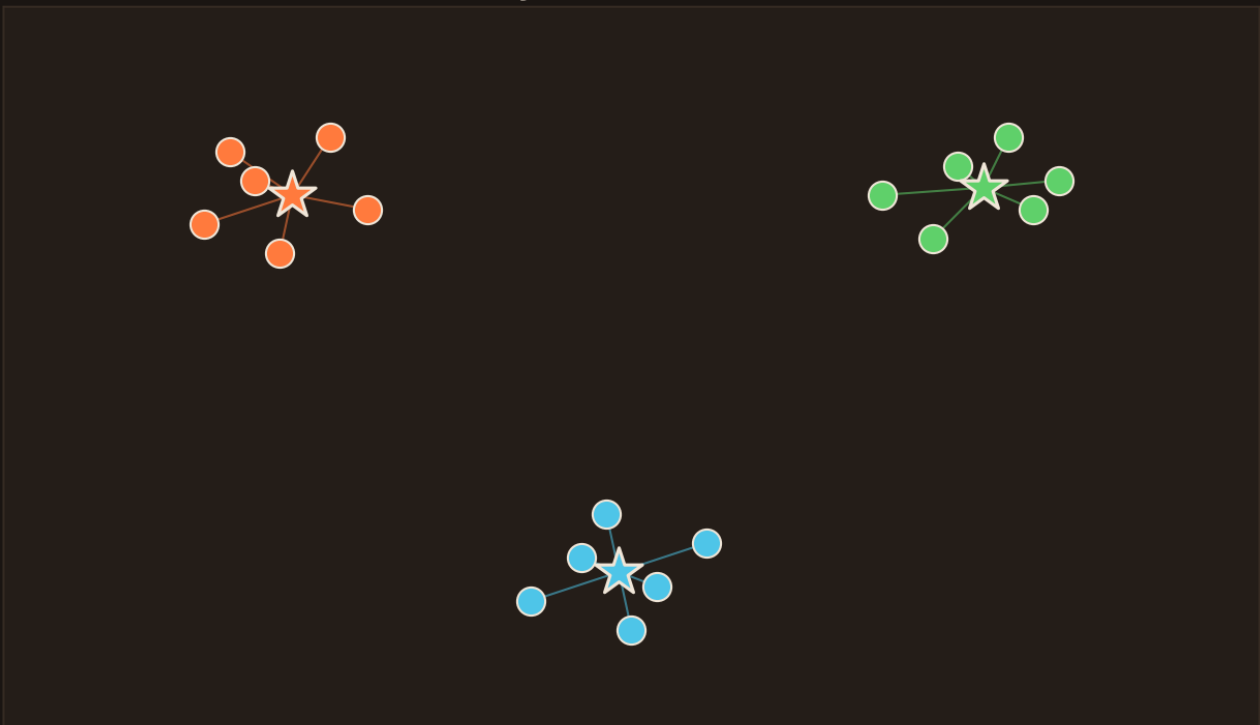


- A cluster by the **cricket pitch**.
- A cluster under the **tree**.

Your eye spots the groups instantly — but *how would a machine*, with no labels, do the same? K-Means is a clever, almost childlike trick to find these natural clumps. And "K" just means **how many groups** you're looking for (here, $K = 3$).

Step 1: drop K spots, everyone joins the nearest

**Step 1: drop K "teacher-spots" (stars).
Each kid joins the NEAREST star.**



The teacher drops K "teacher-spots" (the stars). Every kid walks to the NEAREST star. Now each star has a rough group around it.

The teacher does something simple. She picks **K spots** on the playground ($K = 3$ here) and calls out: "Everyone, come stand near the closest spot to you!"

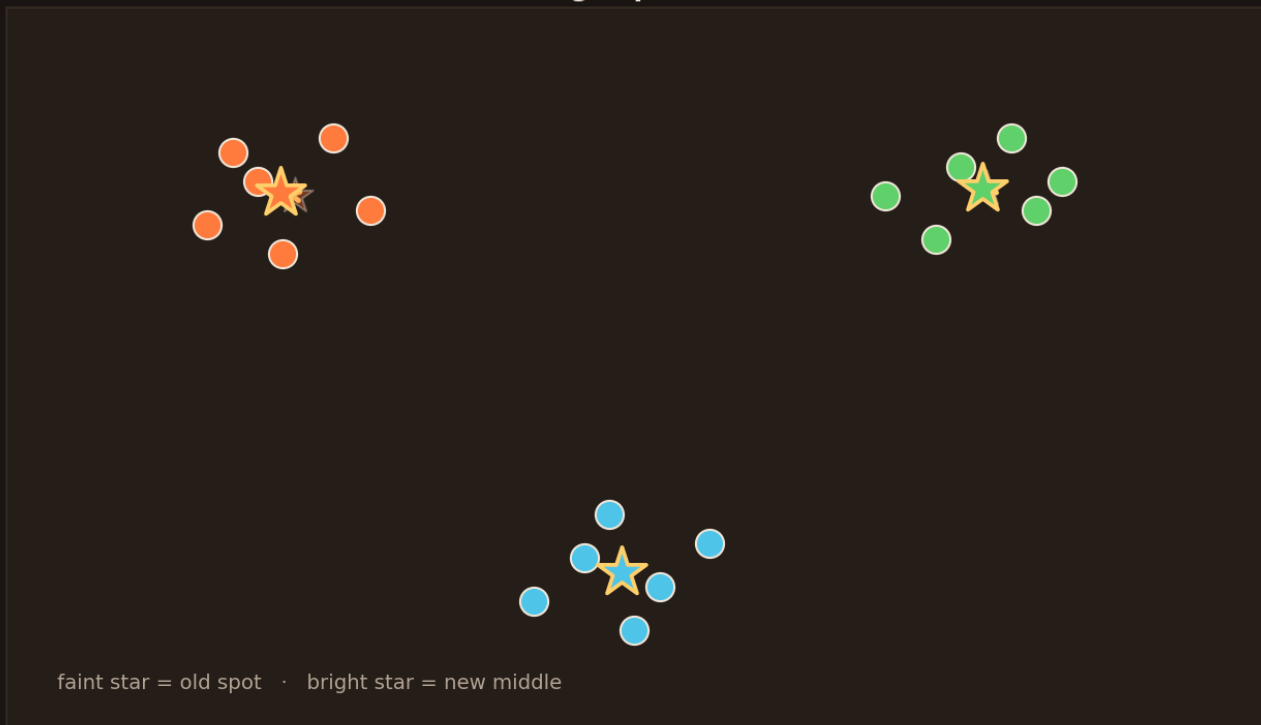
- Drop **K center-spots** (the stars) — at first, just anywhere.
- Every kid looks around and joins the **nearest** star.
- Now each star has a rough group standing around it.

That's the first move: **assign each point to its nearest center**. Same "closest wins" idea you saw in KNN — distance decides. But the first spots were random guesses, so the groups aren't perfect yet. That's what the next step fixes.



Step 2: each spot moves to the middle of its group

Step 2: each star moves to the MIDDLE of its group
(the average spot of its kids)



Each star slides from its old spot (faint) to the MIDDLE of its group (bright) — the average position of all its kids. The center moves to where its crowd actually is.

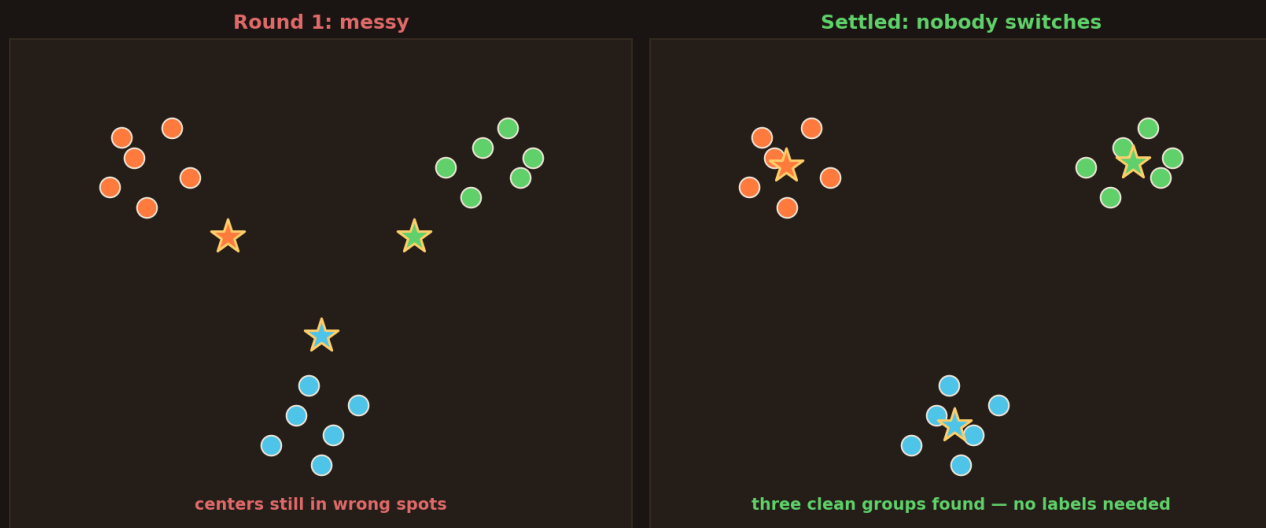
The teacher looks at each group and thinks: *"My spot isn't quite in the middle of my kids."* So each star **moves to the average position** of its group — the true middle.

- Look at all the kids around a star.
- Move the star to their **middle** (the average spot).
- The center is now sitting right in the heart of its crowd.

This is the heart of K-Means: **move each center to the middle of its group**. The word "Means" in the name literally means **average** — the center is the average of its kids. Assign to nearest, then move to the middle. Two simple moves.

Step 3: repeat until nobody switches





Left: round 1 is messy — the centers are still in wrong spots and groups are mixed. Right: after repeating, the centers settle and three clean groups appear — with no labels at all.

Here's the magic: you just **repeat those two steps**. Assign to nearest → move to middle → assign again → move again...

- Each round, kids may **switch** to a closer star, and stars slide to new middles.
- Slowly, everything settles — kids stop switching, stars stop moving.
- **When nobody switches, you're done** — the natural groups are found.

Left picture: messy first round, centers in the wrong place. Right picture: after a few rounds, three clean groups appear — swings, cricket, tree — and **nobody had to label a single kid**. The machine found the structure by itself. That's the beauty of unsupervised learning.

The "formula" — but really simple

There's no scary equation. K-Means is trying to make each group **as tight as possible** — kids close to their own center.

Goal: make the total distance from every kid to its own center as SMALL as possible.

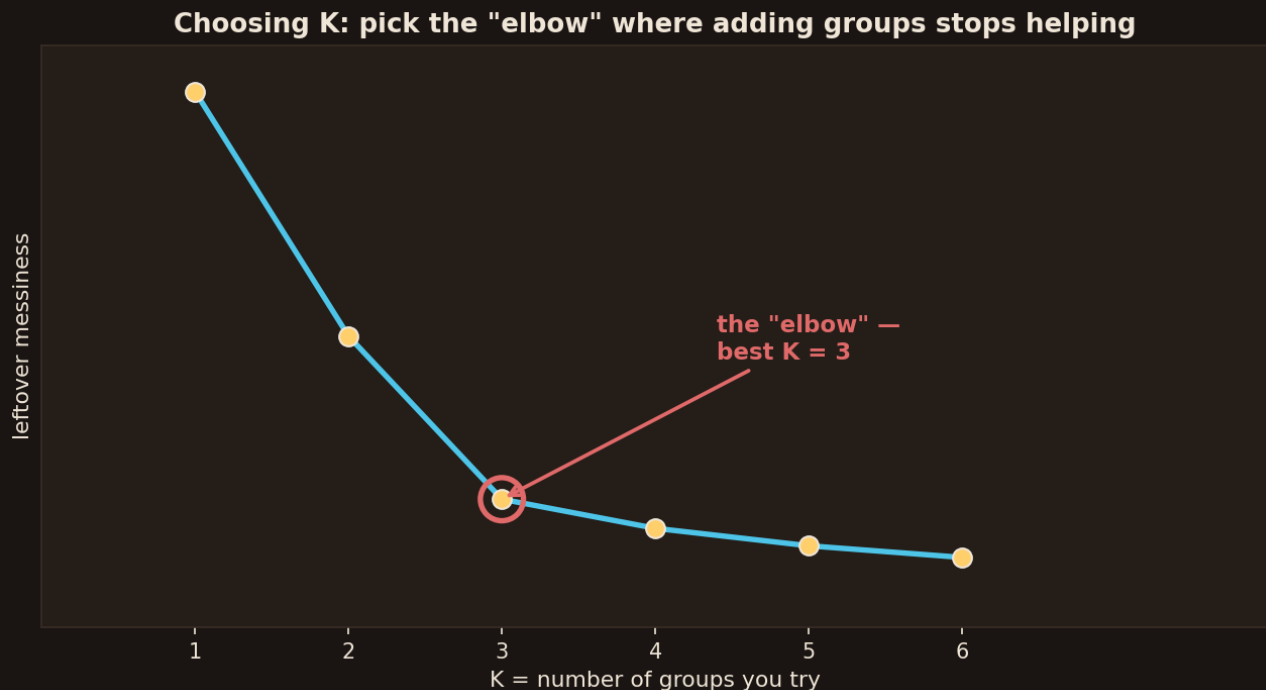
That "total distance" has a plain meaning:

- If a group is **tight** (kids hugging their center) → small distance → good.
- If a group is **spread out** (kids far from their center) → big distance → bad.
- Every round of "assign + move to middle" **shrinks** this total distance a little, until it can't shrink more. That's when it stops.



So the whole engine is: **keep shrinking the distance from kids to their centers.** Assign-nearest shrinks it, move-to-middle shrinks it more, repeat until it won't budge. No calculus needed — just "make the groups tight."

How many groups? Choosing K (the elbow)



Try $K = 1, 2, 3, 4, \dots$ and plot the leftover messiness. It drops fast, then flattens. The sharp bend — the "elbow" — is your best K (here, 3).

One catch: **you have to tell K-Means how many groups to look for.** How do you pick? Try a few values and watch the "leftover messiness":

- **Too few groups** ($K=1$ or 2) → groups are big and loose → lots of messiness.
- **More groups** → tighter, less messiness — but past a point, adding groups barely helps.
- The **"elbow"** — the sharp bend where the curve flattens — is the sweet spot. Here it's **$K = 3$** .

The elbow trick: plot messiness for $K = 1, 2, 3, 4, \dots$ and look for the bend, like an arm's elbow. Before the elbow, more groups help a lot; after it, they barely help. Pick the elbow — that's usually the natural number of groups.

Try it — a few lines



```
from sklearn.cluster import KMeans

model = KMeans(n_clusters=3)    # K = 3 groups
model.fit(X)                   # X = your unlabelled data

print(model.labels_)           # which group each point landed in
print(model.cluster_centers_)  # the final middle of each group
```

`n\_clusters` is your K. Notice there's **no y** (no answer labels) in `fit` — that's what makes it unsupervised. `labels\_` tells you the group each point ended up in; `cluster\_centers\_` are the final star positions.

Where it shines and where it slips

- **Great for finding hidden groups** → customer segments, grouping similar products, sorting news into topics — anywhere you have data but no labels.
- **Fast and simple** → assign + average, repeated. Scales to lots of data.
- **You must pick K** → it can't decide the number of groups for you; use the elbow.
- **Assumes round, similar-size blobs** → if the real groups are long, curvy, or very different sizes, K-Means struggles (it likes neat round clumps).
- **Sensitive to the first spots** → bad starting stars can settle wrongly; the fix is to run it a few times and keep the best (sklearn does this for you).

The one thing to remember: K-Means is your go-to for **finding natural groups when you have no labels** — fast, simple, and widely used. Just remember you choose K (use the elbow), and it works best when the groups are neat, round clumps.

Recap — the 20-second version

- K-Means finds groups with **no labels** (unsupervised learning).
- Like a teacher who **drops K spots** and tells each kid to join the nearest.
- Then each spot **moves to the middle** of its group ("Means" = average).
- **Repeat** assign + move until **nobody switches** — the groups appear.
- Pick the number of groups **K** using the **elbow**.



Next up — Video 14: Principal Component Analysis (PCA). What if your data has TOO many columns to even see? PCA is the magic that squishes hundreds of features down to just two or three — keeping the important shape and throwing away the noise. See you Day 14.

